Newton

Inovação é a nossa tradição.

Estruturas de Dados

Prof. Fabrício Rodrigues Inácio

fabricio.inacio@newtonpaiva.br

Aula passada...

- Estruturas de dados dinâmicas
- Listas encadeadas
- Unidade básica de listas encadeadas
- Operações com listas encadeadas

Operações com listas encadeadas

- Inserir no início – insere um elemento no início da lista.
- Inserir no meio – insere um elemento no meio da lista.
- Inserir no final – insere um elemento no final da lista.
- Lista vazia – verifica se a lista está vazia.
- Elemento do início – mostra o elemento que está no início da lista.
- Elemento do final – mostra o elemento que está no final da lista.
- Remover – elimina um elemento que está na lista ligada.
- Contar nós – verifica quantos elementos existem na lista.
- Mostrar lista – exibe todos os elementos que a lista possui.
- Buscar – verifica se determinado elemento pertence à lista.
- Destruir – elimina todos os elementos da lista ligada.
- Antecessor – retorna o elemento que antecede um determinado elemento.
- Sucessor – retorna o elemento que está inserido após um determinado elemento.

Implementações para listas encadeadas

Várias estruturas de dados podem ser usadas para representar listas lineares, cada uma com vantagens e desvantagens particulares. As duas representações mais utilizadas são:

- Implementação por arranjos;
- Implementação por ponteiros (ou referência).

Implementações com arranjos

- Os elementos são armazenados em um arranjo de tamanho suficiente para armazenar a lista;
- A lista pode ser percorrida em qualquer direção;
- O campo último aponta para a posição seguinte à do último
- elemento da lista;
- O i-ésimo elemento da lista está armazenado na (i-1)-ésima
- posição do arranjo;
- O tamanho do arranjo limita o tamanho máximo permitido
- para a lista.

Implementações com arranjos

Vantagens da implementação por arranjos

- Economia de memória (os apontadores são implícitos nesta estrutura);
- Acesso a qualquer elemento da lista é feito em tempo constante (O(1)).

Desvantagens da implementação por arranjos

- Custo para inserir elementos da lista pode ser linear (O(n)
  - todos os elementos podem precisar ser movidos);
- Custo para remover elementos da lista pode ser O(n);
- Quando não existe previsão sobre o crescimento da lista, o arranjo que define a lista deve ser alocado de forma dinâmica;
- O arranjo pode precisar ser realocado completamente.

Implementações com arranjos

Exercícios práticos

1. Escreva um método de inserção que permita adicionar itens na lista mantendo-se a ordenação (considere que a lista estará ordenada inicialmente ou vazia).

2. Escreva um método que permita aumentar o tamanho da lista.

3. Escreva um método que permita remover um elemento qualquer da lista.

Limitações da implementação com arranjos

Quais os principais problemas de utilizarmos arranjos para implementar listas?

Limitações da implementação com arranjos

Quais os principais problemas de utilizarmos arranjos para implementar listas?

E se a lista aumentar e depois diminuir drasticamente de tamanho?

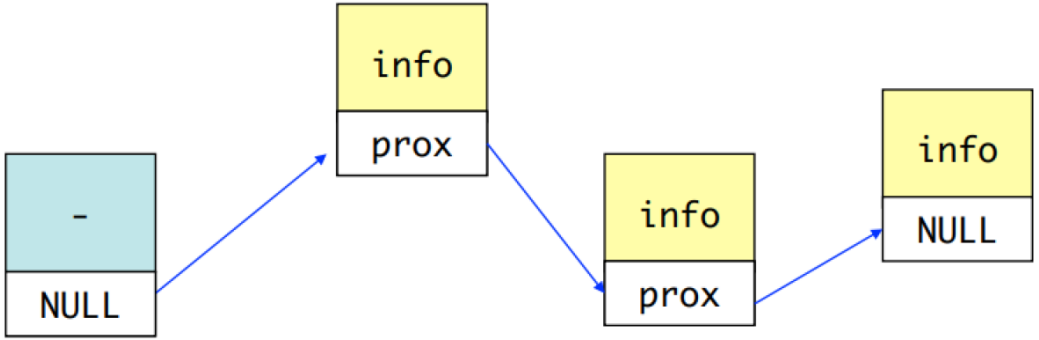
A solução para este problema está na implementação de uma lista encadeada utilizando referências (ou ponteiros).

Implementações com referências

O tamanho não é fixo nem pré-definido;

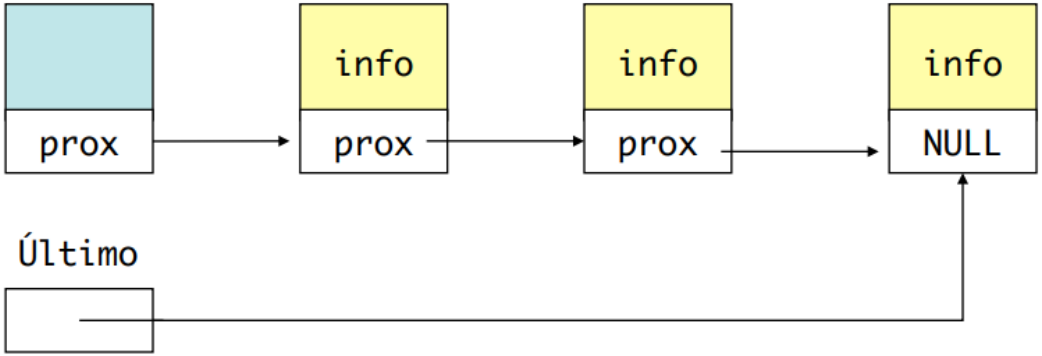
Cada elemento aponta quem é o próximo elemento na lista;

Elementos não estão contíguos na memória;

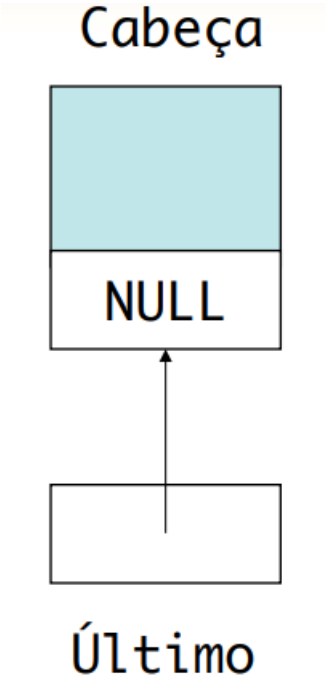


Implementações com referências

Para facilitar a utilização e melhorar o desempenho, as listas podem ter um elemento cabeça, que aponta para o primeiro elemento válido da lista e um apontador para o último endereço da lista.



Criando uma lista vazia

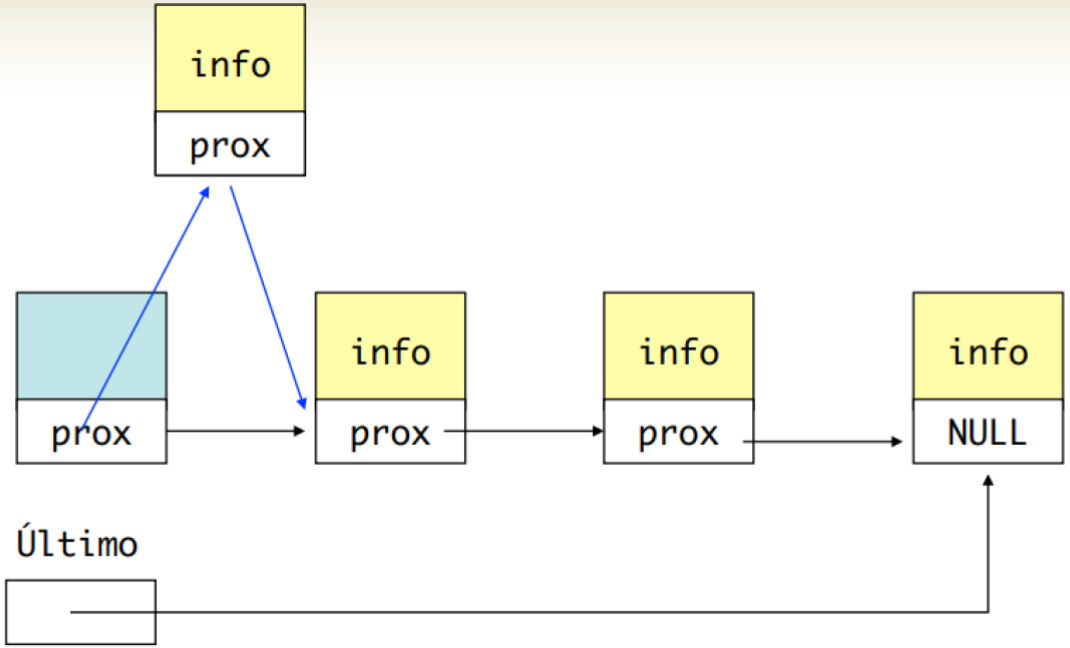


Inserção de elementos na lista

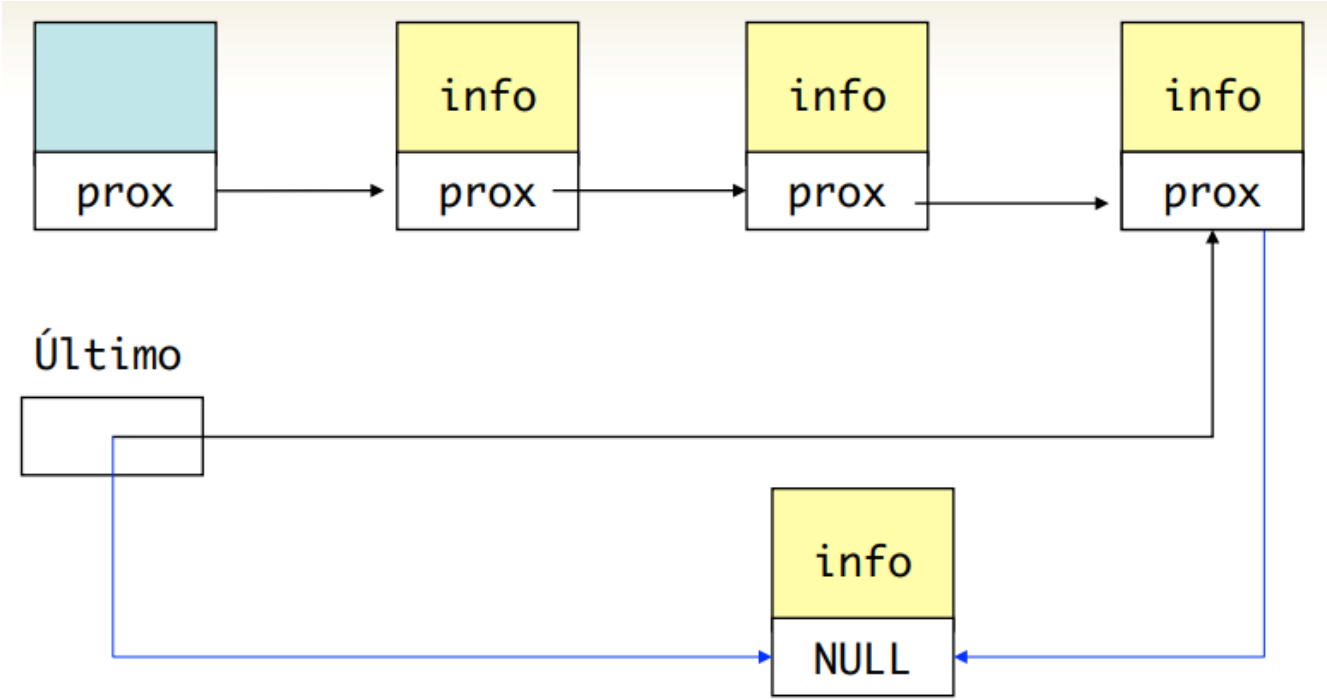
A inserção de dados na lista encadeada pode ocorrer em três pontos:

- No início;
- No final;
- Após um elemento qualquer.

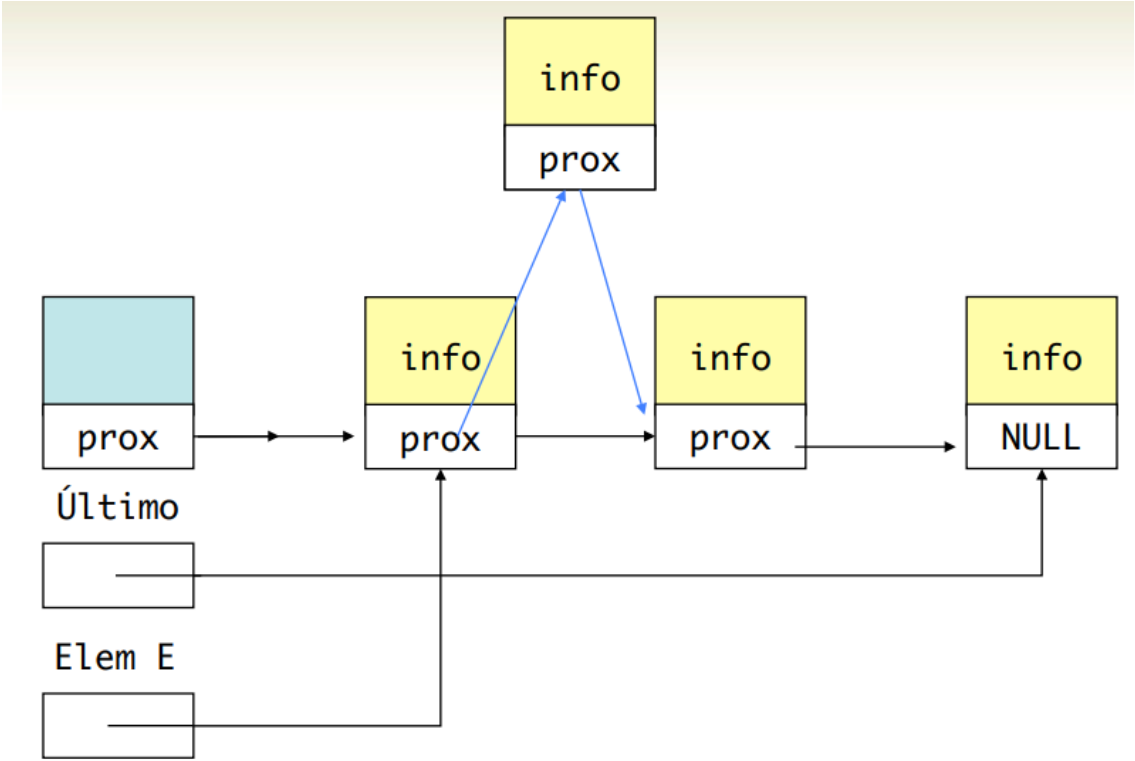
Inserção na primeira posição



Inserção na última posição



Inserção após um elemento

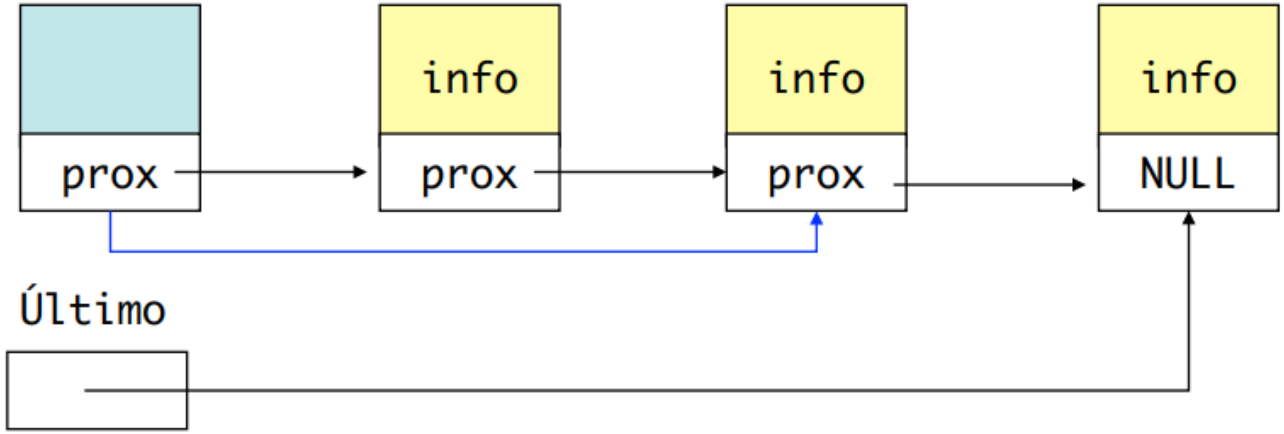


Remoção de elementos da lista

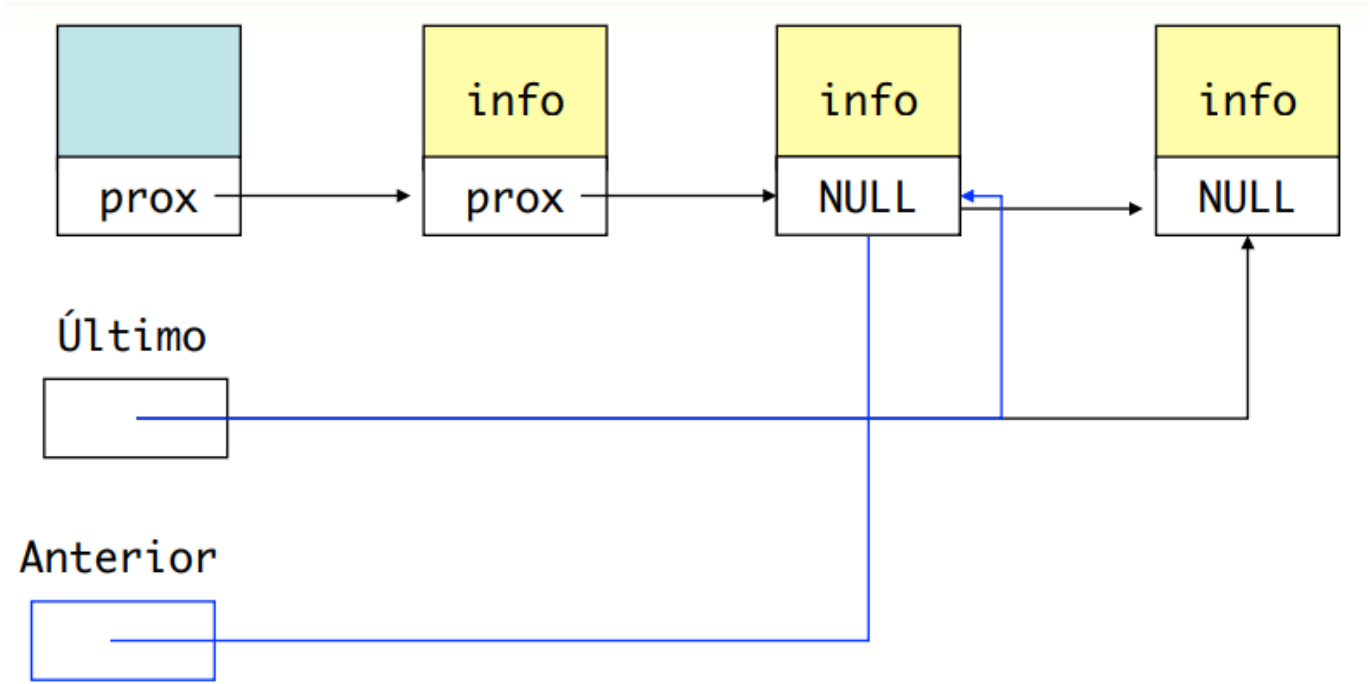
A remoção de dados na lista encadeada pode ocorrer em três pontos:

- No início;
- No final;
- Após um elemento qualquer.

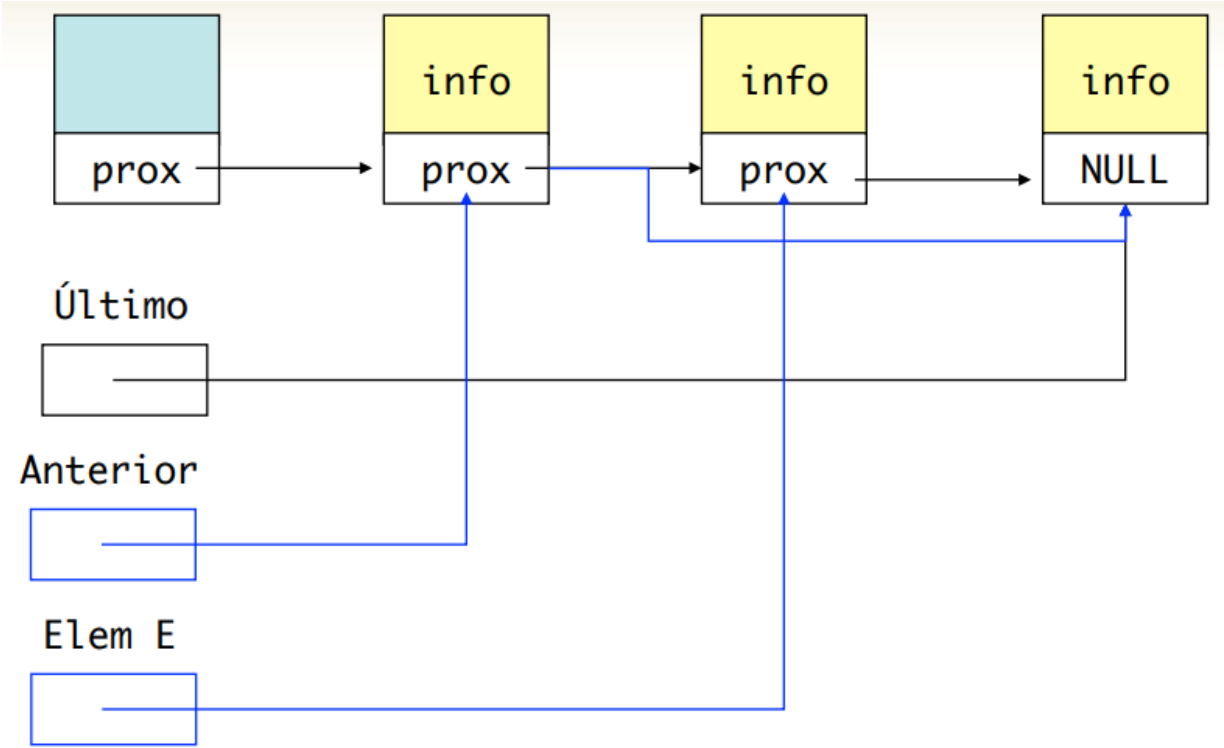
Remoção da primeira posição



Remoção da última posição



Remoção de um elemento específico



Remoção de um elemento específico

Exercícios práticos

1. Implemente uma função de inserção que mantenha a lista sempre ordenada.
2. Implemente uma função para retornar o valor do antecessor de um elemento.
3. Implemente um função para retornar o valor do sucessor de um elemento.